

DevOps Cheat Sheet

Linux, Git, Terraform

A command reference, grouped by what you are trying to do.

Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1	Basic Linux Commands	2
2	Intermediate Linux Commands	2
3	Advanced Linux Commands	2
4	Networking	3
5	File Management and Search	3
6	System Monitoring	3
7	Package Management	4
8	Disk and Filesystem	4
9	Scripting and Automation	4
10	Development and Debugging	4
11	Other Useful Commands	5
12	Basic Git Commands	6
13	Branching and Merging	6
14	Remote Repositories	6
15	Stashing and Cleaning	7
16	Tagging	7
17	Advanced Git Commands	7
18	GitHub CLI	7
19	Terraform	9

1. Basic Linux Commands

Command	Description
pwd	Print the current working directory
ls	List files and directories
cd	Change directory
touch	Create an empty file
mkdir	Create a new directory
rm	Remove files or directories
rmdir	Remove empty directories
cp	Copy files or directories
mv	Move or rename files and directories
cat	Display the content of a file
echo	Display a line of text
clear	Clear the terminal screen

2. Intermediate Linux Commands

Command	Description
chmod	Change file permissions
chown	Change file ownership
find	Search for files and directories
grep	Search for text in a file
wc	Count lines, words and characters in a file
head	Display the first few lines of a file
tail	Display the last few lines of a file
sort	Sort the contents of a file
uniq	Remove duplicate lines from a file
diff	Compare two files line by line
tar	Archive files into a tarball
zip/unzip	Compress and extract ZIP files
df	Display disk space usage
du	Display directory size
top	Monitor system processes in real time
ps	Display active processes
kill	Terminate a process by its PID
ping	Check network connectivity
wget	Download files from the internet
curl	Transfer data from or to a server
scp	Securely copy files between systems
rsync	Synchronise files and directories

3. Advanced Linux Commands

Command	Description
awk	Text processing and pattern scanning
sed	Stream editor for filtering and transforming text
cut	Remove sections from each line of a file
tr	Translate or delete characters

Command	Description
xargs	Build and execute command lines from standard input
ln	Create symbolic or hard links
df -h	Display disk usage in human-readable format
free	Display memory usage
iostat	Display CPU and I/O statistics
netstat	Network statistics; ss is the modern replacement
ifconfig	Configure network interfaces; ip is the modern replacement
iptables	Configure firewall rules
systemctl	Control the systemd system and service manager
journalctl	View system logs
crontab	Schedule recurring tasks
at	Schedule tasks for a specific time
uptime	Display system uptime
whoami	Display the current user
users	List all users currently logged in
hostname	Display or set the system hostname
env	Display environment variables
export	Set environment variables

4. Networking

Command	Description
ip addr	Display or configure IP addresses
ip route	Show or manipulate routing tables
traceroute	Trace the route packets take to a host
nslookup	Query DNS records
dig	Query DNS servers
ssh	Connect to a remote server via SSH
ftp	Transfer files using the FTP protocol
nmap	Network scanning and discovery
telnet	Communicate with remote hosts
nc	Read and write data over networks (netcat)

5. File Management and Search

Command	Description
locate	Find files quickly using a prebuilt database
stat	Display detailed information about a file
tree	Display directories as a tree
file	Determine a file's type
basename	Extract the filename from a path
dirname	Extract the directory part of a path

6. System Monitoring

Command	Description
<code>vmstat</code>	Display virtual memory statistics
<code>htop</code>	Interactive process viewer, an alternative to <code>top</code>
<code>lsof</code>	List open files
<code>dmesg</code>	Print kernel ring buffer messages
<code>uptime</code>	Show how long the system has been running
<code>iostat</code>	Display real-time disk I/O by process

7. Package Management

Command	Description
<code>apt</code>	Package manager for Debian-based distributions
<code>yum/dnf</code>	Package manager for RHEL-based distributions
<code>snap</code>	Manage snap packages
<code>rpm</code>	Manage RPM packages

8. Disk and Filesystem

Command	Description
<code>mount</code>	Attach a filesystem; <code>umount</code> detaches it
<code>fsck</code>	Check and repair filesystems
<code>mkfs</code>	Create a new filesystem
<code>blkid</code>	Display information about block devices
<code>lsblk</code>	List block devices as a tree
<code>parted</code>	Manage partitions interactively

9. Scripting and Automation

Command	Description
<code>bash</code>	Command interpreter and scripting shell
<code>sh</code>	Legacy shell interpreter
<code>cron</code>	Automate recurring tasks
<code>alias</code>	Create shortcuts for commands
<code>source</code>	Execute commands from a file in the current shell

10. Development and Debugging

Command	Description
<code>gcc</code>	Compile C programs
<code>make</code>	Build and manage projects
<code>strace</code>	Trace system calls and signals
<code>gdb</code>	Debug programs
<code>git</code>	Version control system

Command	Description
vim/nano	Text editors for scripting and editing

11. Other Useful Commands

Command	Description
date	Display or set the system date and time
cal	Display a calendar
man	Display the manual for a command
history	Show previously executed commands

12. Basic Git Commands

Command	Description	Example
<code>git init</code>	Initialises a new Git repository in the current directory	<code>git init</code>
<code>git clone</code>	Copies a remote repository to the local machine	<code>git clone https://github.com/user/repo.git</code>
<code>git status</code>	Displays the state of the working directory and staging area	<code>git status</code>
<code>git add</code>	Adds changes to the staging area	<code>git add file.txt</code>
<code>git commit</code>	Records staged changes to the repository	<code>git commit -m "Initial commit"</code>
<code>git config</code>	Configures user settings such as name and email	<code>git config --global user.name "Your Name"</code>
<code>git log</code>	Shows the commit history	<code>git log</code>
<code>git show</code>	Displays detailed information about a specific commit	<code>git show</code>
<code>git diff</code>	Shows changes between commits, the working directory and the staging area	<code>git diff</code>
<code>git reset</code>	Unstages changes or resets commits	<code>git reset HEAD file.txt</code>

13. Branching and Merging

Command	Description	Example
<code>git branch</code>	Lists branches or creates a new one	<code>git branch feature-branch</code>
<code>git checkout</code>	Switches between branches or restores files	<code>git checkout feature-branch</code>
<code>git switch</code>	Switches branches, the modern alternative to <code>checkout</code>	<code>git switch feature-branch</code>
<code>git merge</code>	Combines changes from one branch into another	<code>git merge feature-branch</code>
<code>git rebase</code>	Moves or combines commits from one branch onto another	<code>git rebase main</code>
<code>git cherry-pick</code>	Applies specific commits from one branch to another	<code>git cherry-pick <commit-hash></code>

14. Remote Repositories

Command	Description	Example
<code>git remote</code>	Manages remote repository connections	<code>git remote add origin https://github.com/user/repo.git</code>
<code>git push</code>	Sends local commits to a remote repository	<code>git push origin main</code>
<code>git pull</code>	Fetches and merges changes from a remote repository	<code>git pull origin main</code>
<code>git fetch</code>	Downloads changes from a remote without merging them	<code>git fetch origin</code>

Command	Description	Example
<code>git remote -v</code>	Lists the URLs of the configured remotes	<code>git remote -v</code>

15. Stashing and Cleaning

Command	Description	Example
<code>git stash</code>	Temporarily saves changes that are not yet committed	<code>git stash</code>
<code>git stash pop</code>	Applies stashed changes and drops them from the stash list	<code>git stash pop</code>
<code>git stash list</code>	Lists all stashes	<code>git stash list</code>
<code>git clean</code>	Removes untracked files from the working directory	<code>git clean -f</code>

16. Tagging

Command	Description	Example
<code>git tag</code>	Creates a tag for a specific commit	<code>git tag -a v1.0 -m "Version 1.0"</code>
<code>git tag -d</code>	Deletes a tag	<code>git tag -d v1.0</code>
<code>git push</code>	Pushes tags to a remote repository	<code>git push origin --tags</code>

17. Advanced Git Commands

Command	Description	Example
<code>git bisect</code>	Finds the commit that introduced a bug	<code>git bisect start</code>
<code>git blame</code>	Shows which commit and author last modified each line	<code>git blame file.txt</code>
<code>git reflog</code>	Shows a log of where the branch tips have been	<code>git reflog</code>
<code>git submodule</code>	Manages external repositories as submodules	<code>git submodule add https://github.com/user/repo.git</code>
<code>git archive</code>	Creates an archive of the repository files	<code>git archive --format=zip HEAD > archive.zip</code>
<code>git gc</code>	Cleans up unnecessary files and optimises the repository	<code>git gc</code>

18. GitHub CLI

Command	Description	Example
<code>gh auth login</code>	Logs into GitHub from the command line	<code>gh auth login</code>

Command	Description	Example
<code>gh repo clone</code>	Clones a GitHub repository	<code>gh repo clone user/repo</code>
<code>gh issue list</code>	Lists issues in a GitHub repository	<code>gh issue list</code>
<code>gh pr create</code>	Opens a pull request	<code>gh pr create --title "New Feature"</code>
<code>gh repo create</code>	Creates a new GitHub repository	<code>gh repo create my-repo</code>

19. Terraform

Command	Description
<code>terraform --help</code>	Display general help for the Terraform CLI
<code>terraform init</code>	Initialise the working directory and download the provider plugins the configuration needs
<code>terraform validate</code>	Check the configuration files for syntax errors and internal inconsistencies
<code>terraform plan</code>	Produce an execution plan showing what Terraform will do to reach the desired state
<code>terraform apply</code>	Apply the changes required to reach the desired state, prompting for approval first
<code>terraform show</code>	Display the state, or a saved plan, in human-readable form
<code>terraform output</code>	Display the output values defined in the configuration
<code>terraform destroy</code>	Destroy the infrastructure the configuration manages, prompting for confirmation
<code>terraform refresh</code>	Update the state file from the real infrastructure without applying any change
<code>terraform taint</code>	Mark a resource for recreation on the next apply, even if nothing about it changed
<code>terraform untaint</code>	Remove the tainted status from a resource
<code>terraform state</code>	Manage the state file: move resources between modules, remove them, inspect them
<code>terraform state list</code>	List every resource tracked in the state file
<code>terraform import</code>	Bring existing infrastructure under Terraform management
<code>terraform graph</code>	Generate a graph of the resources and their dependencies
<code>terraform providers</code>	List the providers the current configuration requires

On the backend: The backend is configured in a `backend` block inside `terraform { }`, not from the command line. It decides where the state file lives: locally by default, or remotely in S3, Azure Blob Storage, Terraform Cloud and others. `terraform init` is what reads it and migrates existing state.